

1 Megoldási ötletek és magyarázat

A probléma lényege, hogy egy n hosszúságú számsorból egy k szélességű mátrixot alakítunk ki, majd meghatározzuk egy speciális alakzat (az ábrán látható *L-alakú* régió) maximális összegét.

1.1 Kulcsfontosságú megfigyelések

1. A bemeneti számsor elemeit k szélességű mátrixba rendezzük sorfolytonosan. Ha n nem osztható k -val, az utolsó sor hiányzó celláit nullákkal pótoljuk (ezt biztosítja az `offset` változó kezelése).
2. A probléma megoldásának alapja a **2D prefix összeg** (más néven integrált kép) adatszerkezet. Egy $[0..i] \times [0..j]$ téglalap összege:

$$S(i, j) = a[i][j] + S(i - 1, j) + S(i, j - 1) - S(i - 1, j - 1)$$

Ez lehetővé teszi bármely téglalap összegének $O(1)$ időben történő kiszámítását.

3. Az `OFFSET_MAX` és a `k - r` eltolás azért szükséges, hogy a prefix összeg számítás során ne kelljen speciális eseteket kezelni az indextúllépések miatt.

1.2 Az algoritmus működése

Az algoritmus három fő lépésből áll:

1. Prefix összeg tábla felépítése

```
for (int i = 0; i <= lastLine; ++i)
    for (int j = 0; j < k; ++j)
    {
        int ind = getCoords(i, j);
        if (i)
            a[ind] += a[getCoords(i - 1, j)];
        if (j)
            a[ind] += a[getCoords(i, j - 1)];

        if (i && j)
            a[ind] -= a[getCoords(i - 1, j - 1)];
    }
```

2. Első konfiguráció: klasszikus L-alak

Az első dupla ciklus az összes lehetséges pozcióban kiszámítja az L-alakzat összegét:

$$\text{eredmény} = \text{lineSum} + \text{colSum} - \text{intersectSum}$$

Ahol:

- **lineSum:** a $k - 1$ -edik oszlop q db sorának összege az $[i - q + 1..i]$ intervallumban
- **colSum:** a **lastLine**-odik sor p db oszlopának összege az $[j - p + 1..j]$ intervallumban
- **intersectSum:** a két régió metszetének összege (ezt kétszer számolnánk, ezért vonjuk ki)

3. Második konfiguráció: horizontális L-alak

A második ciklus egy alternatív elrendezést vizsgál, ahol az összeget a teljes összegből kiindulva módosítjuk a hiányzó és hozzáadott részek figyelembevételével.

1.3 Idő- és térkomplexitás

- **Térkomplexitás:** $O(k \cdot \lceil n/k \rceil) = O(n + k)$ — a mátrix mérete arányos a bemeneti mérettel
- **Időkomplexitás:** $O(n)$ — a prefix összeg felépítése $O(n)$, a keresési ciklusok szintén $O(n)$

A megoldás tehát lineáris időben fut, ami elegendő a korlátok teljesítéséhez.

1.4 Megvalósítási részletek

- Az $s(i, j)$ segédfüggvény a negatív indexeket nullával kezeli, így egységesíti a prefix összeg elérését
- A **getCoords** függvény a 2D indexeket 1D-ba konvertálja: $x \cdot k + y$
- A $p > k$ eset kezelése biztosítja, hogy ne lépjük túl a mátrix szélességét

A megoldás lényege, hogy a 2D prefix összeg adatszerkezet segítségével az L-alakzatok összegét konstans időben ki tudjuk számítani, így a teljes algoritmus lineáris komplexitású.